

Welcome to the AI Specialist Curriculum

This roadmap is designed to build a deep theoretical understanding and high-performance engineering skills in Artificial Intelligence. The curriculum covers classic state-space searches, deep learning, computer vision architectures, advanced Natural Language Processing, Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), AI Agent architectures, containerized model serving, and critical AI Safety guardrails.

Table of Stages / Content Map:

- Stage 0 — Prerequisites (Python, NumPy, SQL, Linux, Basic Math)
- Stage 1 — Artificial Intelligence Fundamentals (PEAS, DFS/BFS, UCS, A*, Minimax, Logic)
- Stage 2 — Mathematics for AI (Linear Algebra, Calculus, Convex Optimization, Loss Functions)
- Stage 3 — Neural Networks & Deep Learning (Backpropagation, Optimizers, Regularization)
- Stage 4 — Computer Vision (CNNs, ResNet, Object Detection YOLO, Segmentations, ViTs)
- Stage 5 — Natural Language Processing (Embeddings, LSTM, Self-Attention, Transformers)
- Stage 6 — Large Language Models (LLMs) (Pretraining, Fine-tuning, RLHF, Quantization, LoRA)
- Stage 7 — Generative AI (VAEs, GANs, Diffusion models, Multimodal Vision-Language models)
- Stage 8 — Retrieval-Augmented Generation (RAG) (Chunking, Vector Databases, Reranking, Evaluation)

- Stage 9 — AI Agents (Tool calling, planning, reflection, ReAct, multi-agent systems)
- Stage 10 — AI Engineering (APIs, Model serving, FastAPI, Docker, GPU fundamentals, Caching)
- Stage 11 — AI Safety & Responsible AI (Hallucinations, Jailbreaks, Guardrails, Governance)
- Stage 12 — 20 Progression Projects (5 Beginner, 5 Inter., 5 Adv., 5 Production-grade)
- Stage 13 — AI Specialist Interview Preparation & System Design Case Studies

Stage 0 — Prerequisites

Minimum competencies required before attempting neural network layers and state search spaces.

Python & Core Math

[MUST KNOW] Comfortable implementing classes, decorators, OOP. Linear Algebra: matrix multiplication, transposes, dimensions. Calculus: derivatives and partial derivatives.

Collections & NumPy

[MUST KNOW] Vectorized calculations, array broadcasts, dot products, matrix slicing without using loops.

SQL & Git

[MUST KNOW] Retrieving datasets from databases, merging tables, and committing/pushing production code splits.

Linux & Command Line

[SHOULD KNOW] Navigating paths, configuring environment variables, running python scripts via shell, and basic pip packages management.

Stage 1 — Artificial Intelligence Fundamentals

Classic algorithmic search and representation paradigms.

Intelligent Agents & PEAS

[MUST KNOW] Agent: perceives environment through sensors, acts via actuators. PEAS: Performance measure, Environment, Actuators, Sensors. Defines task environments.

Uninformed Search (BFS, DFS, UCS)

[MUST KNOW] BFS: level order, optimal for unweighted. DFS: depth first, low memory. UCS (Uniform Cost Search): expands node with lowest path cost, optimal for weighted edges.

Heuristic Search (A*, Greedy Best First)

[MUST KNOW] Greedy: expands node with lowest $h(n)$. A* Search: minimizes $f(n) = g(n) + h(n)$. Optimal if heuristic $h(n)$ is admissible (never overestimates actual cost) and consistent.

Adversarial Search (Minimax, Alpha-Beta)

[MUST KNOW] Minimax: decision rule for multi-player games. Alpha-Beta Pruning: decreases nodes evaluated by pruning branches that cannot impact final decisions (bounds alpha/beta).

Knowledge Representation & Logic

[SHOULD KNOW] Propositional Logic (evaluates absolute True/False variables) and First-Order Logic (incorporates predicates, objects, and quantifiers like 'for all' and 'exists').

Key Search Algorithms Comparison

Algorithm	Time Complexity	Space Complexity	Optimality
BFS	$O(b^d)$	$O(b^d)$ (high memory)	Yes (if edge costs are 1)
DFS	$O(b^m)$	$O(b * m)$ (low memory)	No
UCS	$O(b^{1 + \text{floor}(C^*/e)})$	$O(b^{1 + \text{floor}(C^*/e)})$	Yes (if edge costs $\geq e > 0$)
A*	$O(b^d)$ (exponential)	$O(b^d)$ (keeps all in memory)	Yes (if heuristic is admissible)

Stage 2 — Mathematics for AI

The mathematical engine powering optimization and representation.

Linear Algebra & Derivatives

[MUST KNOW] Matrix dimensions, dot products, projections, eigenvalues/eigenvectors, partial derivatives, Jacobian matrices, chain rule.

Convexity & Optimization

[SHOULD KNOW] Convex functions (local minimum is global minimum), Gradient Descent (optimizing weights sequentially in direction of steepest descent).

Loss Functions

[MUST KNOW] Mean Squared Error (regression), Binary Cross-Entropy (binary classification), Categorical Cross-Entropy (multi-class predictions).

Stage 3 — Neural Networks & Deep Learning

Deep representations constructed via chain rule backpropagation.

Perceptron & Activation Functions

[MUST KNOW] Perceptron: $\text{output} = \text{activation}(w \cdot x + b)$. Activation functions: Sigmoid (0 to 1), Tanh (-1 to 1), ReLU ($\max(0, x)$, avoids gradients vanishing), LeakyReLU (prevents dead neurons).

Forward & Backpropagation

[MUST KNOW] Forward: compute layer-by-layer activations. Backpropagation: calculates gradients of loss function with respect to weights using chain rule, propagating error backward.

Optimizers (Momentum, Adam)

[MUST KNOW] SGD with Momentum: adds velocity vector to smooth out oscillations. Adam (Adaptive Moment Estimation): maintains adaptive learning rates using first and second moments of gradients.

Regularizations (Dropout, Batch Norm)

[MUST KNOW] Dropout: randomly deactivates neurons during training to prevent co-adaptation. Batch Normalization: standardizes layer inputs per batch to accelerate convergence.

Stage 4 — Computer Vision

Architectures designed to extract spatial features from image pixels.

CNN Layers (Convolutions, Pooling)

[MUST KNOW] Convolution: kernel sliding over pixels to create feature maps. Padding: keeping dimensions constant. Stride: step size. Pooling (Max/Average): spatial downsampling.

ResNet & Transfer Learning

[MUST KNOW] ResNet: skip connections (identity shortcuts) allowing gradients to flow directly, resolving vanishing gradients in very deep networks. Transfer learning: using pre-trained weights (e.g. ImageNet).

Object Detection (YOLO, Faster R-CNN)

[SHOULD KNOW] YOLO (You Only Look Once): single-stage detector, predicts boxes and probabilities in one forward pass (high speed). Faster R-CNN: two-stage detector using Region Proposal Networks (high accuracy).

Vision Transformers (ViT)

[ADVANCED] Applying self-attention to patches of images. Split image into grid patches, flatten them, add positional encoding, and feed into a standard transformer encoder.

Stage 5 — Natural Language Processing (NLP)

Representing and sequence modelling textual parameters.

Classical NLP & Embeddings

[MUST KNOW] Tokenization, lemmatization, Bag of Words, TF-IDF. Word2Vec/GloVe: dense low-dimensional vector representations capturing semantic similarities.

Recurrent Networks (LSTM, GRU)

[MUST KNOW] RNN: sequential tokens scan. LSTM (Long Short-Term Memory): maintains cell state, controlled via Input, Forget, and Output gates, resolving short-term memory decays. GRU: simplified gates (Reset/Update).

Self-Attention & Multi-Head Attention

[MUST KNOW] Self-Attention: calculates token relationships by computing Query (Q), Key (K), and Value (V) dot products: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$. Multi-Head: performs projections in parallel.

Stage 6 — Large Language Models (LLMs)

Billions of parameters models optimized via next-token predictions.

Pretraining & Next-Token Prediction

[MUST KNOW] Generative pretraining on massive web-scale corpora using causal language modeling (predicting the next token given context).

Fine-Tuning (Instruction & RLHF)

[MUST KNOW] Instruction Tuning: training model on prompt-response pairs. RLHF (Reinforcement Learning from Human Feedback): aligning models using a reward predictor and PPO or DPO.

PEFT & Parameter Efficiency (LoRA)

[MUST KNOW] LoRA (Low-Rank Adaptation): freezes model weights and inserts low-rank decomposition matrices (A and B) into attention layers, reducing trainable parameters by 99%.

Quantization (QLoRA)

[SHOULD KNOW] Reducing precision (e.g. from FP16 to INT4/INT8) using double quantization and Pageable Optimizers, enabling LLM fine-tuning on consumer-grade GPUs.

Stage 7 — Generative AI

Generative models synthesizing structured pixel, audio, or text parameters.

Variational Autoencoders (VAEs)

[SHOULD KNOW] Compresses images into a latent space (mean and log-variance) and reconstructs them, regularized via Kullback-Leibler (KL) divergence.

Generative Adversarial Networks (GANs)

[SHOULD KNOW] Two-player minimax game: Generator synthesizes fake samples; Discriminator attempts to classify real vs. fake. Trained in parallel.

Diffusion Models

[MUST KNOW] Generates high-fidelity images by reversing a forward noise process. Slowly adds noise to image, then trains a U-Net model to predict and subtract noise step-by-step.

Stage 8 — Retrieval-Augmented Generation (RAG)

Injecting external verified knowledge documents into LLM prompts without retraining weights.

Vector Databases & Embeddings

[MUST KNOW] Converting chunks of documents into floating-point vectors. Storing in databases (Chroma, FAISS, Pinecone) for similarity searches.

Chunking Strategies & Retrieval

[MUST KNOW] Split documents logically (e.g. recursive character splits with 500-token size, 50-token overlap). Retrieve top-K chunks using cosine similarity.

Reranking & Hybrid Search

[SHOULD KNOW] Hybrid: combine keyword BM25 search with dense vector search. Reranking: using Cross-Encoder models to re-evaluate semantic matches, filtering top chunks to fit context windows.

RAG Evaluation & Failure Modes

[MUST KNOW] Evaluation metrics (faithfulness, answer relevance, context recall) using frameworks like Ragas. Failure modes: out-of-context chunks, LLM hallucination despite retrieve.

Stage 9 — AI Agents

Autonomous entities combining reasoning, tool use, memory, and multi-agent structures.

Agent Reasoning & ReAct

[MUST KNOW] ReAct (Reason + Action) loop: Thought (reason about state), Action (call tool), Observation (read tool output), Repeat. Prevents flat prompt actions.

Tool Calling & State Memory

[MUST KNOW] LLMs outputting JSON matching tool schemas. Memory: maintaining conversational state, summarizations, or key-value memory banks.

Multi-Agent Orchestration

[ADVANCED] Splitting complex workflows into dedicated agents (e.g. Planner, Researcher, Writer) communicating via message buses (LangGraph).

Stage 10 — AI Engineering & Serving

Deploying, hosting, and optimizing large neural architectures.

API Serving & FastAPI

[MUST KNOW] Exposing models via REST endpoints. Creating input-validation schemas using Pydantic inside FastAPI.

Containerization (Docker)

[MUST KNOW] Packaging environment variables, CUDA requirements, and python scripts inside a lightweight Docker container for cloud scaling.

GPU Optimization & Inference

[SHOULD KNOW] Optimizing memory: batching queries, setting up prompt caching, and using vLLM (PagedAttention) to scale concurrent serving throughput.

Stage 11 — AI Safety & Responsible AI

Securing deployments against adversarial prompt injections and jailbreaks.

Hallucination & Bias Mitigation

[MUST KNOW] Evaluating output alignments, checking truthfulness benchmarks, and implementing system prompts to bound response structures.

Prompt Injection & Jailbreaking

[MUST KNOW] Adversarial prompts attempting to override system instructions. Secure via input guardrails (Llama Guard) and token validation filters.

AI Governance & Guardrails

[SHOULD KNOW] Ensuring data privacy compliance (GDPR, HIPAA) and running post-generation toxic content analysis.

Stage 12 — 20 Progression Projects

Practical portfolios designed to validate production-level AI engineering capabilities.

Level 1 — Beginner Projects (5 Projects)

- 1. Image Classifier (PyTorch + CIFAR-10 - training CNN from scratch, calculating validation loss)
- 2. Custom A* Path Finder (Python - search space visualization, grid-obstacle coordinate heuristic maps)
- 3. Text Sentiment Classifier (PyTorch + LSTM on movie reviews, embedding layer setups)
- 4. Perceptron logic gate simulator (Scratch code - plotting linear decision boundaries for AND/OR gates)
- 5. TF-IDF document search engine (Python - sparse representation and cosine similarity retrieval)

Level 2 — Intermediate Projects (5 Projects)

- 1. Transfer Learning ResNet Image Classifier (PyTorch - freezing backbone weights, custom output heads)
- 2. YOLO Object Detector Pipeline (YOLOv8 + OpenCV - local video stream bbox drawing)
- 3. Next-Character GRU Text Generator (PyTorch - character-level tokenization, temperature scaling)
- 4. Fine-Tune Llama-3-8B using LoRA (Unsloth/HuggingFace - dataset formatting, save adapters)
- 5. Basic RAG document Q&A; (ChromaDB + OpenAI API + LangChain - basic chunk-and-retrieval pipeline)

Level 3 — Advanced Projects (5 Projects)

- 1. Vision Transformer from scratch (PyTorch - grid patching, patch embeddings, multi-head self-attention)
- 2. VAE Image Synthesizer (PyTorch - latent space reparameterization trick, reconstruction loss plotting)
- 3. Advanced RAG with Reranking & Hybrid search (Qdrant + Cohere Rerank + BM25 + Weaviate)
- 4. Autonomous ReAct Agent with Tool calling (LangGraph - state transitions, SQL tools access)
- 5. Containerized GPU model server (FastAPI + Docker + vLLM - hosting Llama-3, benchmark latency)

Level 4 — Portfolio/Production Projects (5 Projects)

- 1. Enterprise RAG Pipeline (Document ingestion pipeline, chunking evaluation, semantic search + LLM Guardrails)
- 2. Multi-Agent Software Development Team (Coder agent, tester agent, manager agent orchestrating code fixes)
- 3. Fine-Tuned Domain-Specific LLM (Instruction tuning via QLoRA, alignment with DPO, deployed to production)
- 4. Real-time Video Segmentation Stream (U-Net segmentation serving on edge GPUs, low latency optimizing)
- 5. Secure AI Assistant Gateway (FastAPI proxy with prompt-injection scanners and cost/token caches)

Stage 13 — AI Specialist Interview Preparation

Common technical evaluation cases and target responses.

Q: Deep Learning: Explaining Vanishing Gradients

Vanishing gradients occur when backpropagated gradients shrink exponentially as they flow backward through deep layers, halting weight updates in early layers. Fixes: 1) ReLU activation, 2) ResNet skip connections, 3) Batch Normalization, 4) Xavier/He weight initializations.

Q: LLMs: Intuition of self-attention scaling factor

Scaling query-key dot products by $1/\sqrt{d_k}$ prevents the values from growing extremely large in magnitude, which would push the softmax function into regions with extremely small gradients (causing vanishing gradient issues during backprop).

Q: RAG System Design: How to design a legal document search engine?

Architecture: 1) Extract PDFs using OCR, 2) Split using recursive layout-aware chunking, 3) Embed chunks using a domain-specific model (e.g. legal-BERT), 4) Save to vector database (Qdrant), 5) Use hybrid search + Cohere reranker to fetch top-5 chunks, 6) Feed to LLM with system prompt limiting context answers, 7) Evaluate correctness via Ragas.

Q: AI Safety: Describe prompt injection mitigation strategies.

Mitigate by: 1) Separating system commands from user text using XML tags, 2) Passing user text through a classifier (Llama Guard) before LLM generation, 3) Using strict JSON outputs schema via outlines/Pydantic to constrain model output spaces.

Recommended Timeline Plans & Checklists

Structured timetables designed for AI Specialist career paths.

Duration	Target Focus	Weekly Milestones
3-Month Plan	Core Math & Deep Learning	Week 1-4: Linear algebra, calculus, search algorithms. Week 5-8: PyTorch, backpropagation, MLP models. Week 9-12: CNN basics, classical NLP, and Level 1 projects.
6-Month Plan	NLP, Vision & LLMs	Months 1-3: Math & PyTorch. Month 4: Object detection (YOLO), transfer learning. Month 5: Transformers self-attention, tokenization, fine-tuning. Month 6: Level 2 projects.
12-Month Plan	Generative AI & Engineering	Months 1-6: Deep learning and NLP foundations. Month 7-8: Diffusion models, Generative AI. Month 9-10: Vector databases, Advanced RAG, AI Agents, LangGraph. Month 11: FastAPI serving, Docker, GPU optimization, Level 3/4 projects. Month 12: Interview preparation.

Final AI Specialist Competency Checklist

- [] Can write backpropagation formulas using multi-variable chain rule derivatives.
- [] Understand the mechanics of skip connections in ResNet and self-attention in Transformers.
- [] Can fine-tune an LLM on custom datasets using LoRA/QLoRA in Hugging Face.
- [] Can design and deploy a production RAG pipeline using vector DBs and rerankers.
- [] Can build state-managed multi-agent workflows using LangGraph.
- [] Can serve models inside a FastAPI endpoint packaged inside a Docker container.
- [] Understand how to evaluate and mitigate prompt injections and hallucinations.